

Database Design

Overview

The backend uses PostgreSQL through async SQLAlchemy and Alembic. The first migration is ``20260527_0001_create_initial_secure_messaging_schema.py``.

The schema supports:

- User authentication
- Refresh-token rotation
- Public device key relay
- Public one-time prekey relay
- Opaque encrypted direct message relay
- Automatic pending blockchain proof metadata storage
- Audit/security event logging

The backend refuses database sessions unless ``DATABASE_URL`` uses the ``postgresql+asyncpg://`` scheme.

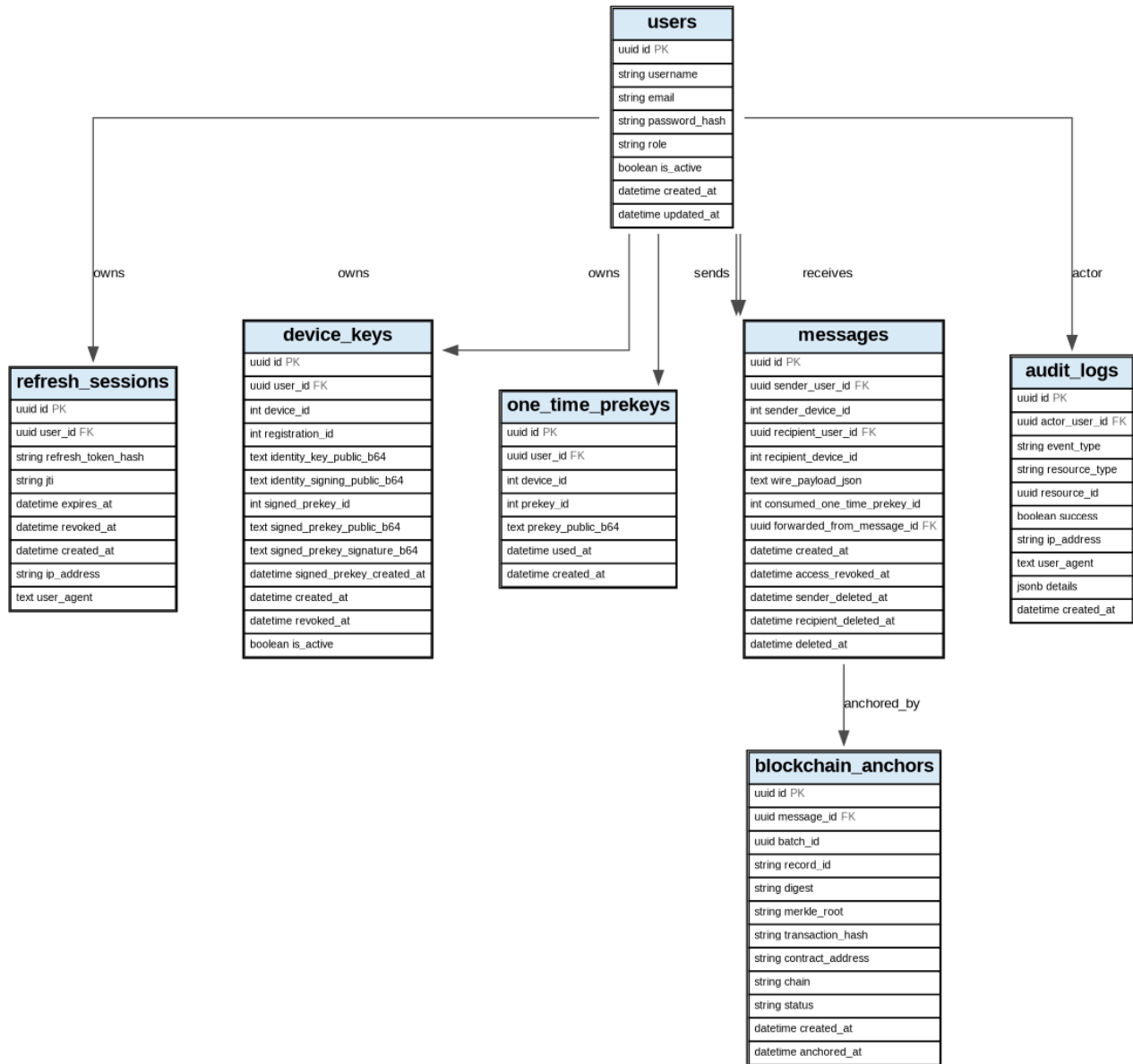


Figure 1: Entity relationship diagram rendered from the Mermaid ERD block.

`alembic\_version` is also created by Alembic to track the current migration revision.

Tables

`users`

Purpose: account identity and authentication metadata.

Important columns:

- `id`: UUID primary key
- `username`: unique user handle
- `email`: unique normalized email
- `password_hash`: Argon2id PHC string
- `role`: currently defaults to `user`
- `is_active`: active/inactive account flag
- `created_at`, `updated_at`: timestamps

Security relevance:

- Stores password hashes only, never plaintext passwords.
- Unique constraints on `username` and `email` support safe duplicate handling.
- `is_active` is checked during login and Bearer token authentication.

Indexes/constraints:

- `uq_users_username`
- `uq_users_email`
- `ix_users_username`
- `ix_users_email`
- `ix_users_is_active`

`refresh_sessions`

Purpose: server-side refresh-token rotation, logout, expiry, and replay detection.

Important columns:

- `user_id`: owner
- `refresh_token_hash`: HMAC-SHA256 wrapped as `hmac_sha256:<digest>`
- `jti`: session identifier
- `expires_at`: refresh session expiry
- `revoked_at`: set when token is rotated or logged out

- `ip\_address`, `user\_agent`: non-secret audit/context fields

Security relevance:

- Raw refresh tokens are never stored.
- Refresh flow locks active session rows and revokes the old session before creating a new one.
- Replay of an old refresh token fails because `revoked\_at` is set.

Indexes/constraints:

- `uq\_refresh\_sessions\_refresh\_token\_hash`
- `uq\_refresh\_sessions\_jti`
- `ix\_refresh\_sessions\_user\_id`
- `ix\_refresh\_sessions\_expires\_at`
- `ix\_refresh\_sessions\_revoked\_at`

`device\_keys`

Purpose: public key bundle storage for each user device.

Important columns:

- `user\_id`
- `device\_id`
- `registration\_id`
- `identity\_key\_public\_b64`
- `identity\_signing\_public\_b64`
- `signed\_prekey\_id`
- `signed\_prekey\_public\_b64`
- `signed\_prekey\_signature\_b64`
- `signed\_prekey\_created\_at`
- `is\_active`, `revoked\_at`

Security relevance:

- Stores public key material only.
- Does not store private keys, root keys, chain keys, ratchet state, or session state.
- Active/non-revoked checks are required before prekeys can be uploaded or bundles returned.

Indexes/constraints:

- `uq\_device\_keys\_user\_id\_device\_id`

- `ix\_device\_keys\_user\_id`
- `ix\_device\_keys\_user\_id\_is\_active`
- `ix\_device\_keys\_revoked\_at`

`one\_time\_prekeys`

Purpose: public one-time prekey pool per user/device.

Important columns:

- `user\_id`
- `device\_id`
- `prekey\_id`: logical client-side/public prekey identifier
- `prekey\_public\_b64`
- `used\_at`
- `created\_at`

Security relevance:

- Public prekey material only.
- `used\_at` is set when the prekey bundle endpoint hands out a prekey.
- The message send route accepts `consumed\_one\_time\_prekey\_id`, but the backend cannot prove the ciphertext cryptographically used that prekey.

Indexes/constraints:

- `uq\_one\_time\_prekeys\_user\_id\_device\_id\_prekey\_id`
- `ix\_one\_time\_prekeys\_user\_id\_device\_id`
- `ix\_one\_time\_prekeys\_user\_id\_device\_id\_used\_at`

`messages`

Purpose: direct 1:1 encrypted relay message storage.

Important columns:

- `sender\_user\_id`
- `sender\_device\_id`
- `recipient\_user\_id`
- `recipient\_device\_id`
- `wire\_payload\_json`
- `consumed\_one\_time\_prekey\_id`
- `forwarded\_from\_message\_id`

- `access\_revoked\_at`
- `sender\_deleted\_at`
- `recipient\_deleted\_at`
- `deleted\_at`

Security relevance:

- `wire\_payload\_json` is an opaque encrypted libsignal-v1 payload string.
- No plaintext `body`, `content`, or `plaintext` column exists.
- Sender identity is derived from the authenticated user in service logic, not client body data.
- Forward provenance is stored as a nullable self-reference controlled by the backend after original-message access is verified.
- Recipient visibility checks exclude revoked, recipient-deleted, and deleted rows.
- Sender visibility checks exclude sender-deleted and deleted rows.

Indexes/constraints:

- `ix\_messages\_recipient\_user\_id\_device\_id\_created\_at`
- `ix\_messages\_sender\_user\_id\_created\_at`
- `ix\_messages\_forwarded\_from\_message\_id`
- `ix\_messages\_access\_revoked\_at`
- `ix\_messages\_sender\_deleted\_at`
- `ix\_messages\_recipient\_deleted\_at`
- `ix\_messages\_deleted\_at`

`blockchain\_anchors`

Purpose: integrity proof metadata associated with encrypted messages or future batches.

Important columns:

- `message\_id`
- `batch\_id`
- `record\_id`
- `digest`
- `merkle\_root`
- `transaction\_hash`
- `contract\_address`
- `chain`

- `status`
- `created\_at`
- `anchored\_at`

Security relevance:

- Stores hashes and transaction metadata only.
- Message send and forward create pending anchors automatically.
- `record\_id` is derived as `keccak256("message:" + message\_id)` for message anchors.
- `digest` is derived from a canonical encrypted message record, including opaque `wire\_payload\_json`, forwarding lineage when present, and metadata only.
- No plaintext, private keys, raw tokens, or ratchet state are stored in anchor rows.
- No FastAPI request handler submits blockchain transactions; the backend blockchain worker confirms pending anchors.

Indexes:

- `ix\_blockchain\_anchors\_message\_id`
- `ix\_blockchain\_anchors\_batch\_id`
- `ix\_blockchain\_anchors\_record\_id`
- `ix\_blockchain\_anchors\_digest`
- `ix\_blockchain\_anchors\_merkle\_root`
- `ix\_blockchain\_anchors\_transaction\_hash`
- `ix\_blockchain\_anchors\_status`

`audit\_logs`

Purpose: security event evidence for authentication, key relay, message actions, and denied access attempts.

Important columns:

- `actor\_user\_id`
- `event\_type`
- `resource\_type`
- `resource\_id`
- `success`
- `ip\_address`
- `user\_agent`

- ``details``
- ``created_at``

Security relevance:

- Details are allowlisted and sanitized by ``audit_service``.
- Passwords, tokens, key material, and ``wire_payload_json`` are not stored in audit details.
- Denied object access is recorded without exposing the protected object.

Indexes:

- ``ix_audit_logs_actor_user_id``
- ``ix_audit_logs_event_type``
- ``ix_audit_logs_created_at``
- ``ix_audit_logs_success``
- ``ix_audit_logs_resource_type_resource_id``

``alembic_version``

Purpose: Alembic-managed migration tracking table.

Security relevance:

- Confirms the deployed database schema revision.
- Final deployment validation checks this revision before the API starts.

Sensitive Data Summary

Stored:

- Argon2id password hashes
- HMAC refresh-token hashes
- JWT/refresh secrets in environment configuration only
- Public key material
- Public one-time prekeys
- Encrypted opaque message relay payloads
- Message metadata
- Blockchain anchor record IDs, digests, roots, transaction metadata, and statuses
- Audit metadata

Not stored:

- Plaintext passwords

- Raw refresh tokens
- Private keys
- Signal ratchet/session state
- Plaintext message bodies
- Group/conversation records

Migration Commands

From `server/backend`:

```
alembic upgrade head
```

```
alembic current
```

```
alembic history
```

Rollback is restricted to controlled local/test environments:

```
alembic downgrade -1
```